

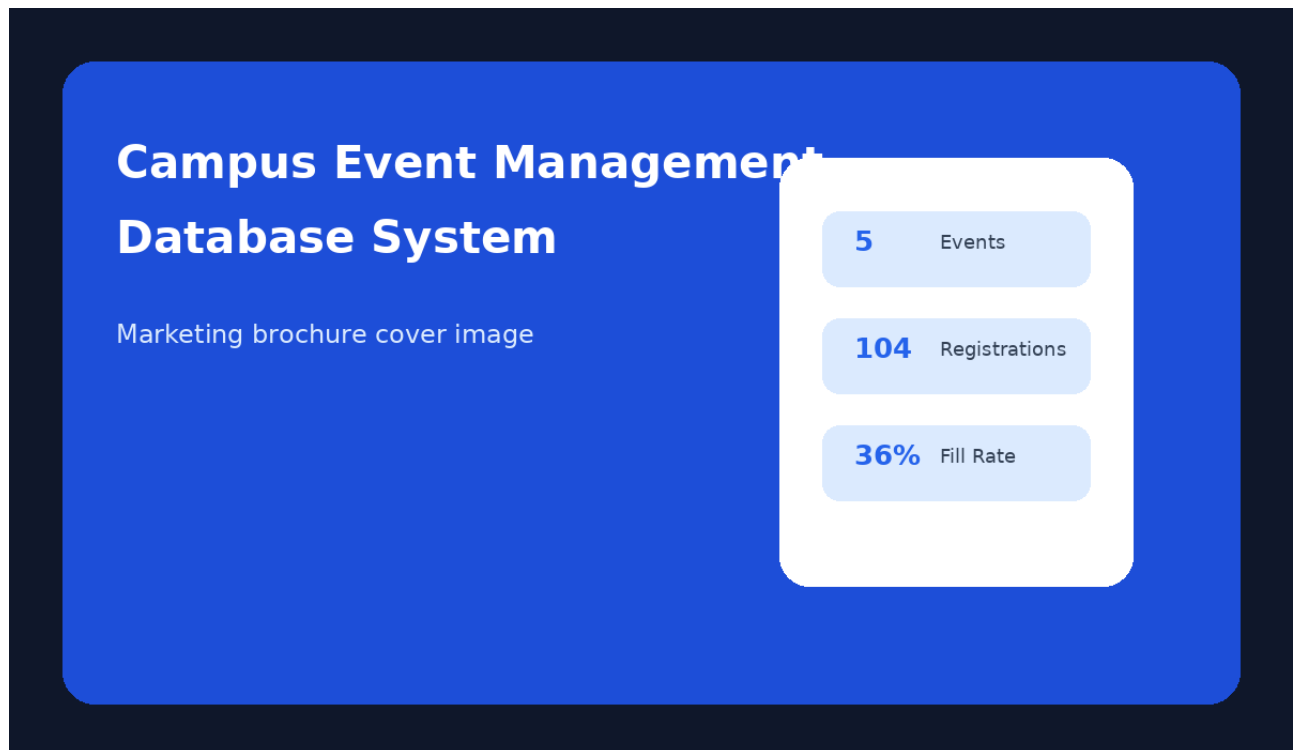
Campus Event Management Database System

System Requirements Documentation - Final Submission

Student Name: Seth Payne

Course: Systems Analysis & Design

Project Type: Browser-Based Event Registration and Tracking Solution



This final report combines problem analysis, system design, implementation planning, testing, maintenance, and deployment preparation for a campus event management solution focused on reducing manual coordination and improving event visibility.

Table of Contents

1. Customer Problem Statement and System Requirements
2. Functional Requirement Specification
3. System Sequence Diagram
4. Activity Diagram
5. User Interface Specification
6. Implementation Planning
7. System Architecture and System Design
8. Algorithms and Data Model Implementation
9. User Interface Design and Implementation
10. Design of Tests
11. Error Handling and Debugging
12. Collaboration and Code Integration
13. Performance Optimization Plan
14. Maintenance and Support Strategy
15. References

1. Customer Problem Statement and System Requirements

Campus staff and student organizations often manage events with a mix of email threads, spreadsheets, paper sign-in sheets, and disconnected calendars. That causes duplicate data entry, inconsistent attendance counts, missed updates, and extra effort when students try to discover events or register quickly.

The goal of this project is to provide one lightweight interface where campus organizers can create events, track registrations, search schedules, and review basic engagement metrics from a single dashboard.

Primary stakeholders

- Student club leaders who need a fast way to create and manage events.
- Campus staff who monitor attendance, capacity, and scheduling conflicts.
- Students who need a simple registration experience with fewer clicks and less confusion.

Core system requirements

- The system shall allow authorized users to create new campus events with organizer, venue, category, date, time, capacity, and description data.
- The system shall allow users to register attendees and automatically update registration totals.
- The system shall prevent duplicate attendee registrations for the same event.
- The system shall support event search and category filtering.
- The system shall display dashboard metrics including event count, registrations, total capacity, and fill rate.
- The system shall persist data for repeated use during the demo session.
- The system shall provide readable feedback messages for both success and failure states.

Non-functional requirements

- Usability: The interface should remain clear and intuitive for first-time users.
- Performance: Normal interactions should respond immediately in the browser.
- Maintainability: Business logic should be separated from presentation logic.
- Portability: The demo should run from a browser and publish easily through GitHub Pages.
- Reliability: Validation and defensive checks should reduce bad data entry.

2. Functional Requirement Specification

The final demo includes a browser-based dashboard with separate flows for event creation, attendee registration, event browsing, and data export. The system emphasizes automation over manual tracking by updating totals, enforcing simple rules, and reducing repetitive work for the user.

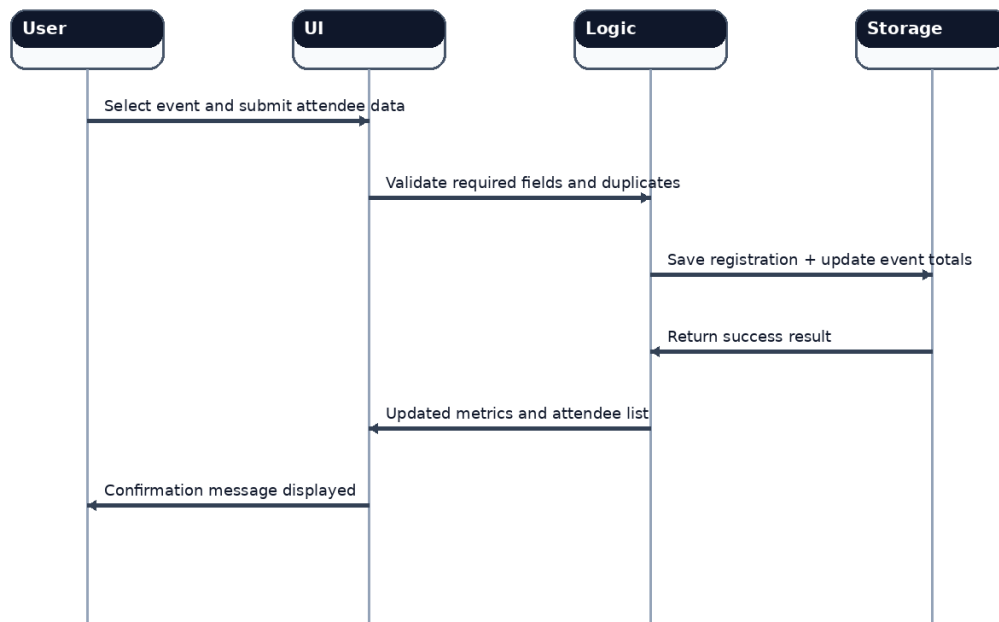
Requirement ID	Description	Priority
----------------	-------------	----------

FR-01	Create and store event records from a structured input form.	High
FR-02	Register attendees and connect them to the selected event.	High
FR-03	Automatically update event registration totals after each successful registration.	High
FR-04	Search and filter events by keyword or category.	Medium
FR-05	Display a recent activity log for quick review and debugging support.	Medium
FR-06	Export collected browser data as JSON for reporting or migration.	Medium

3. System Sequence Diagram

The diagram below shows the attendee registration interaction. The user submits data through the interface, the business layer validates the request, the storage layer persists the data, and the updated result returns to the dashboard automatically.

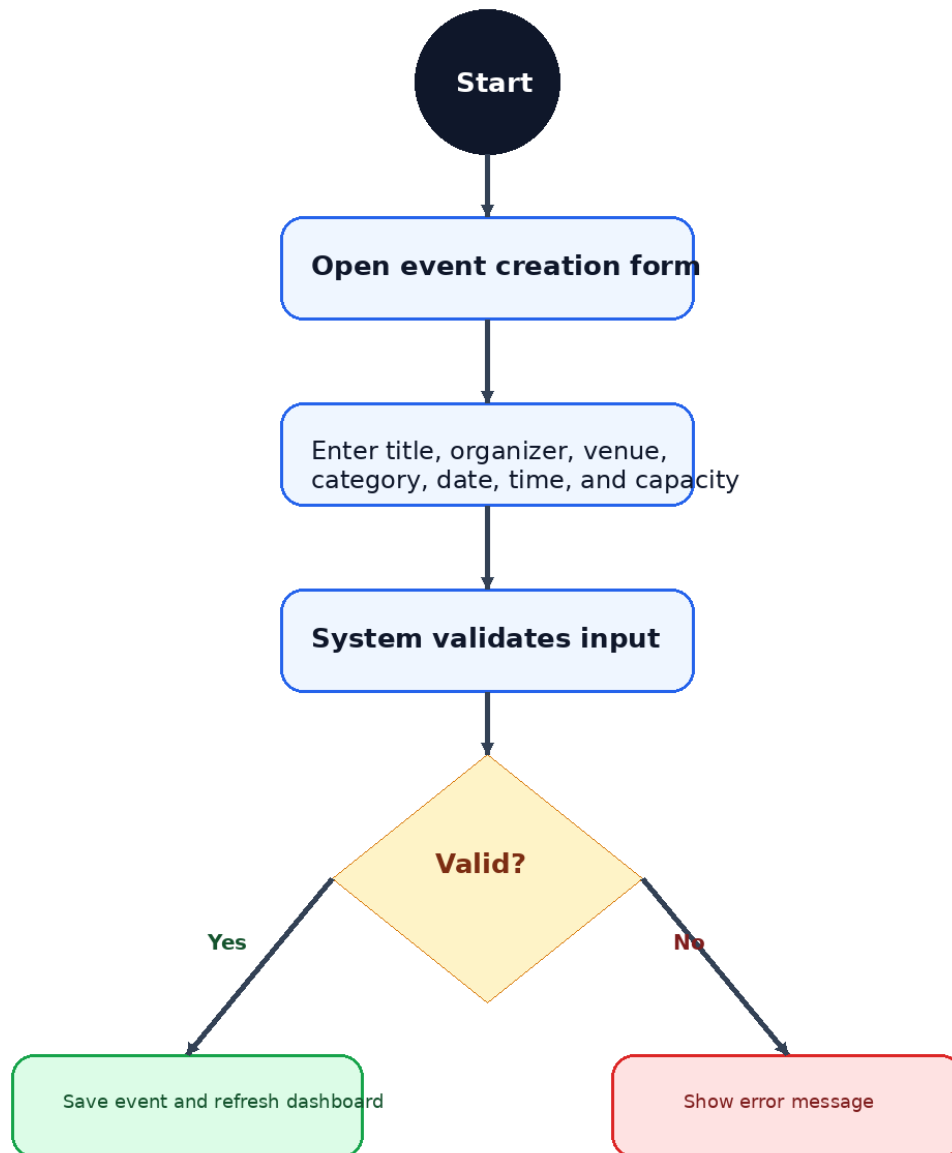
System Sequence Diagram - Register Attendee



4. Activity Diagram

The create-event activity diagram emphasizes how the application validates input before saving data. When required fields are missing, the system displays a clear error instead of storing invalid records.

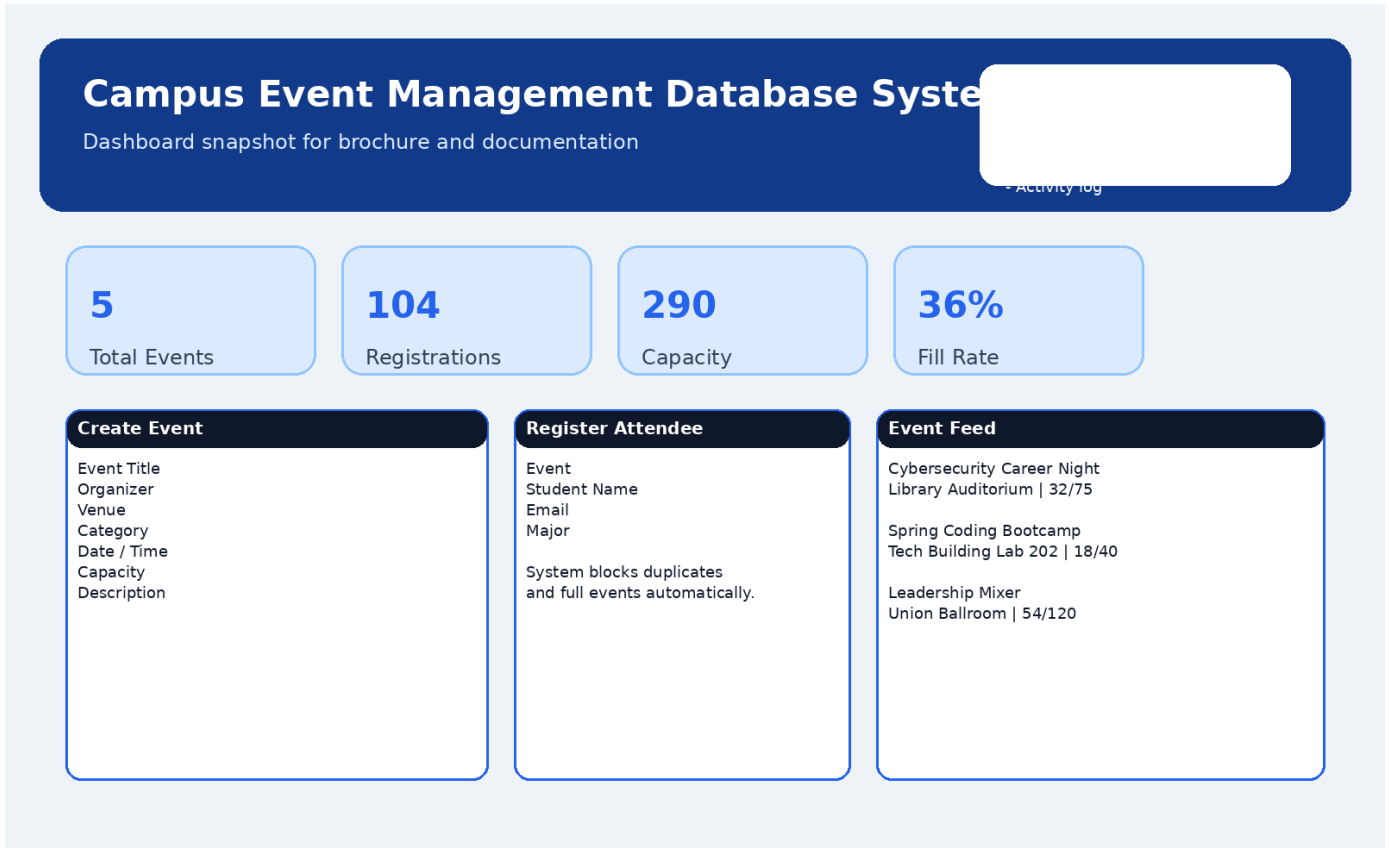
Activity Diagram - Create Event Workflow



5. User Interface Specification

The interface is organized into clear panels that map to the main user tasks. The top of the screen presents a concise problem-focused summary and a dashboard. Below that, users work through two primary forms: event creation and attendee registration. Supporting sections provide search, event browsing, registration tables, and an activity feed.

- Input controls use labeled fields and grouped layouts for readability.
- Dashboard cards surface important status information without navigating away.
- Search and filter controls minimize the number of clicks needed to find relevant events.
- Feedback messages appear near the form where the action occurred.
- Tables and cards use high contrast and generous spacing to improve readability.



6. Implementation Planning

The implementation approach uses phased delivery. Phase 1 focused on defining entities, workflows, and the user interface structure. Phase 2 produced the working browser prototype with local data persistence. Phase 3 added testing assets, export support, maintenance planning, and GitHub-ready packaging for final submission.

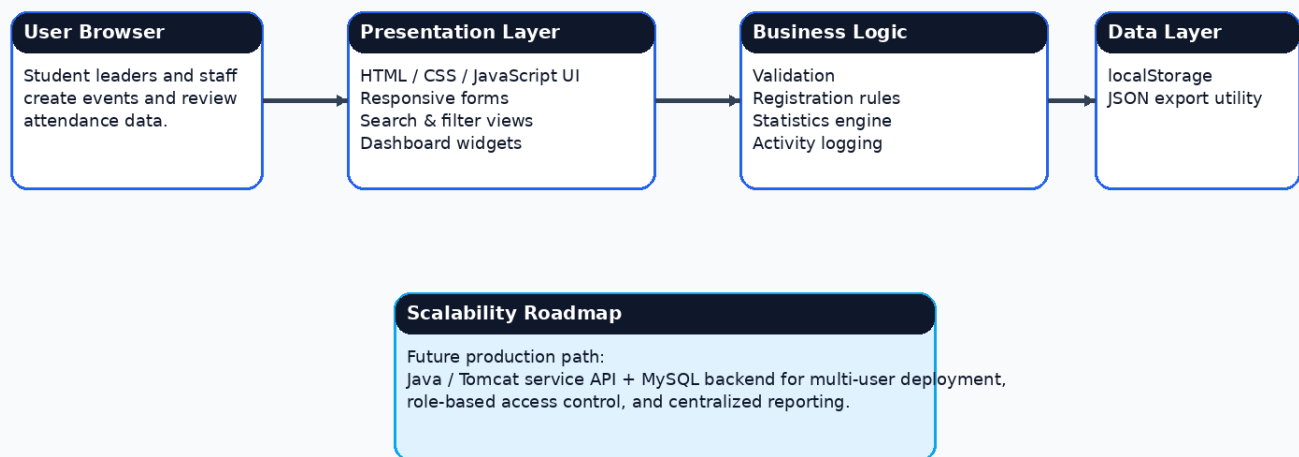
Week	Activity
1	Requirements review, project scope confirmation, and wireframe outline
2	Data model design, ERD, and event/registration workflow planning
3	HTML/CSS/JavaScript implementation of dashboard and forms

4	Validation rules, metrics automation, and persistence logic
5	Unit testing, integration testing, and export utility
6	Documentation, brochure, slides, and deployment preparation

7. System Architecture and System Design

The current deliverable is a lightweight single-page application suitable for local browser execution and GitHub Pages publication. The presentation layer handles forms and rendering, the logic layer processes validation and calculations, and the data layer stores browser state and exports JSON snapshots. A future production path would replace browser storage with a centralized Java plus MySQL service.

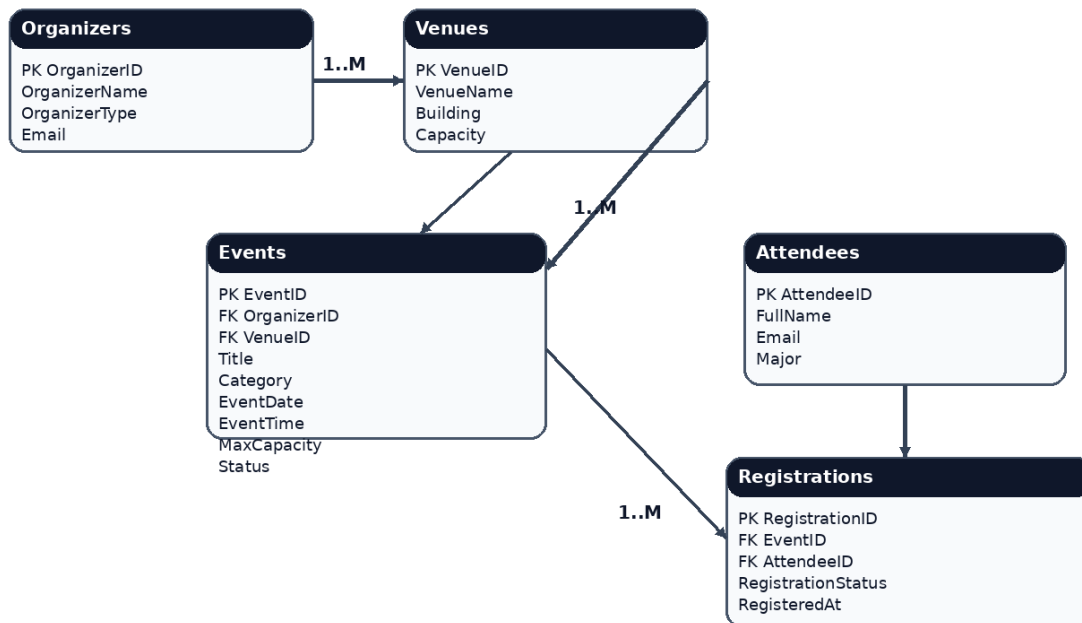
System Architecture



8. Algorithms and Data Model Implementation

The data model uses normalized entities for organizers, venues, events, attendees, and registrations. Registrations act as the bridge table between attendees and events, which supports one attendee joining many events and one event containing many attendees. This reduces redundancy and supports reporting later.

Entity Relationship Diagram



Key algorithms

- Validation algorithm: checks required fields, numeric capacity rules, and basic email format before submission.
- Duplicate detection algorithm: prevents the same email from registering twice for the same event.
- Metrics aggregation algorithm: recalculates totals, capacity, and fill rate from current state values.
- Search and filter algorithm: combines keyword search across multiple text fields with category filtering.

9. User Interface Design and Implementation

The interface uses a responsive card layout and modular JavaScript structure. `logic.js` contains reusable data rules and calculations, while `app.js` handles DOM updates and user actions. This separation supports maintainability, testing, and easier debugging. Styling uses consistent spacing, rounded containers, and high-contrast headings to create a professional but simple presentation.

- Responsive grid layout for desktop and smaller screens.
- Clear section labels that align with the user workflow.
- Visual grouping of related tasks such as creation, registration, and review.
- Persistent activity log to surface automated actions in real time.

10. Design of Tests

Testing was designed at three levels to match the course expectations and demonstrate reliable behavior. The unit tests focus on business rules in isolation, while the integration tests verify a realistic workflow from event

creation to persistence.

Test Level	Purpose	Example Cases
Unit	Verify reusable functions independently	Validate bad input, duplicate registration blocking, metrics recalculation
Integration	Verify interaction across multiple modules	Create event -> register attendee -> reload storage -> verify persistence
System	Manual end-to-end walkthrough	Open dashboard, create event, search event, register student, export data

11. Error Handling and Debugging

The demo uses defensive validation before data changes occur. When a user submits invalid data, the interface returns specific messages instead of silently failing. Activity logging also provides a lightweight debugging trail so that instructors can see what the system did automatically during the demo.

- Required-field validation for event creation.
- Email-format checks for attendee registration.
- Duplicate registration and capacity checks before saving data.
- Activity log records for create, register, and delete actions.
- Structured JavaScript functions that are easier to isolate during debugging.

12. Collaboration and Code Integration

The project package is organized for GitHub-based submission and review. A simple branching workflow is recommended even for a solo project because it supports safer updates and clearer change history. Production-ready work would follow feature branches, pull requests, review, and merge into a protected main branch.

- Main branch for stable deliverables.
- Feature branches for new enhancements or bug fixes.
- Commit messages tied to visible system changes.
- Structured folder layout that maps directly to the instructor's submission checklist.

13. Performance Optimization Plan

The current browser demo performs well because it uses lightweight data structures and client-side rendering. Future optimization opportunities include pagination for large event lists, server-side indexing once a database is introduced, and caching of commonly viewed event summaries.

- Use array filtering only on the current in-memory dataset.
- Reduce repeated DOM updates by re-rendering grouped sections.
- Add database indexes for event dates and registration lookups in the production schema.
- Preserve simple JSON export for easy reporting and migration.

14. Maintenance and Support Strategy

Maintenance planning is important because campus event workflows change over time. The project is structured so new categories, data fields, reporting features, or role-based permissions can be added without rewriting the entire interface. Documentation and testing files make future updates easier to verify.

- Corrective maintenance: fix bugs in validation, rendering, or storage handling.
- Adaptive maintenance: add new event fields, user roles, or reporting outputs.
- Perfective maintenance: improve usability, accessibility, and analytics over time.
- Support materials: README files, brochure, presentation slides, and technical documentation.
- Deployment support: GitHub repository plus optional GitHub Pages demo link for instructor review.

15. References

Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson.